SELF-EVOLVING IMITATION LEARNING

Reel-Learn

Imitate the boring tasks humans do — and evolve, using YouTube reels.

by **Yash Butala** (ML, Apple) & **Ipsita Praharaj** (PM, PayPal) — CMU · IIT KGP

`confusing task` → `watch one reel` → `agent replays it` → `fails once, keeps the fix`

What it is

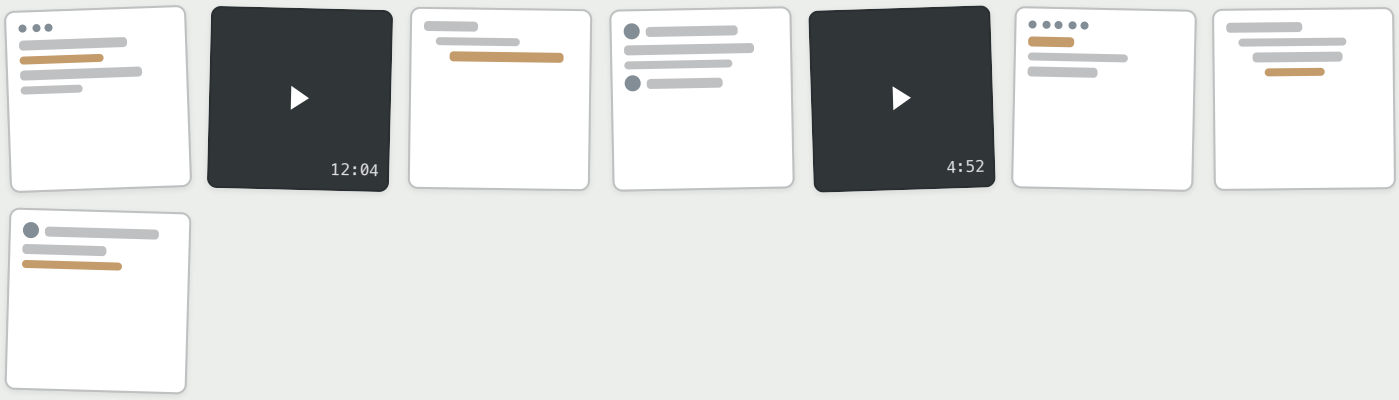
A self-evolving, imitation-learning agent for the harder, more boring tasks nobody wants to do — the ones that need hand-holding through video and visual context, not just instructions.

WHAT SOLVING IT USUALLY LOOKS LIKE

"*why is my battery suddenly dying?*"

"*where did they move this setting*"

"*how do I remove background noise*"



---

01 Current landscape of solving tasks

Boring, not hard — just missing context

Confusing UI, an eight-step workflow, a setting buried in an app you opened once — none of it

Humans already reach for reels — agents should too

Nobody opens docs for a task like this, they open a Short, walked through step by step. If humans

deserves the time it takes. The real problem underneath is context: you don't know this app, this menu, this screen, and nothing on it explains itself.

need that much visual guidance for an unfamiliar task, an agent will need it too.

02 Why the hard tasks matter more

Benchmarks reward what's already easy

OSWorld-style evals score whether a task finished, on tasks a model already knows — precisely the part of the distribution a human never needed a tutorial for.

The tail is where it actually matters

Unfamiliar, multi-step, niche tasks are where a bad plan actually shows itself — and solving one converts it into a reusable skill, not a one-off heuristic.

03 Why not just depend on a teacher for reasoning?

Every attempt starts from zero

Trajectories are generated live for every task — failure is answered with more retries and replanning, not more memory.

Nothing learned ever sticks

A human correction rarely survives past the session it happened in, so the same workflow gets fought through again instead of replayed.

04 Why the reel, specifically

Reels are free, ready-made trajectories

Someone already recorded the task and re-uploads it every time the UI changes — and cut for pacing, a short is already a timestamped sequence of discrete UI actions, closer to a trajectory than to a paragraph of instructions.

The grounding they give isn't optional

Which icon, which toggle, which menu resolves by watching a cursor move, not by reading about it. Holo, the base model underneath this, isn't starting from zero either — walkthroughs just stop it from repeating its own failure cases.

4.1 — Why Audacity, specifically

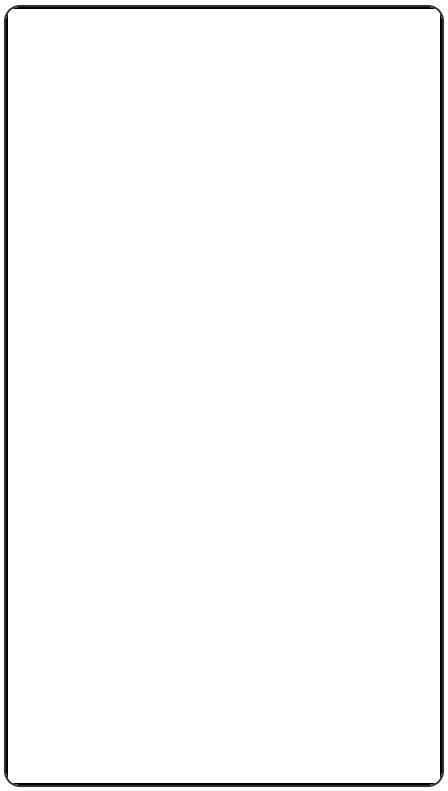
It's not a friendly demo, it's the one where our own base agent struggled. Working from text instructions alone, HComp's agent took 46 clicks and still hadn't finished the job. Handed the reel instead, it solved it directly.

TEXT INSTRUCTIONS ONLY

THE REEL

~~46 clicks~~

Step by step, from text alone — and still not done.



solved, first pass

Same task, same agent — shown the tutorial instead of told about it.

Bigger models don't close this gap alone — reasoning can't see a UI it's never watched. What closes it is a `REEL` human in the loop.

One worked example, end to end — including where it broke: the pipeline case study →

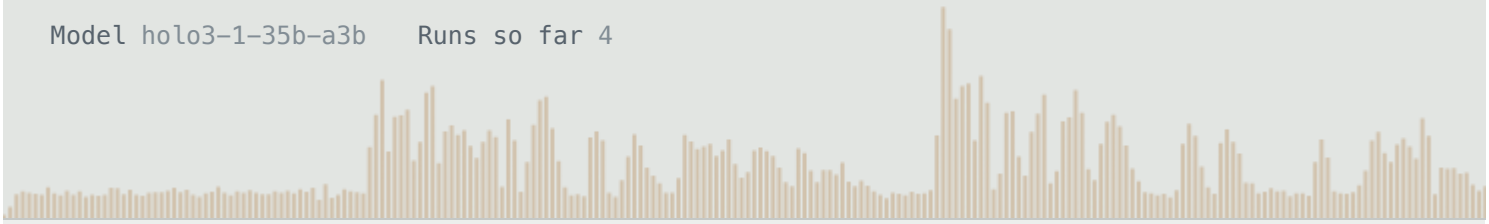
CASE STUDY — COMPUTER-USE AGENTS

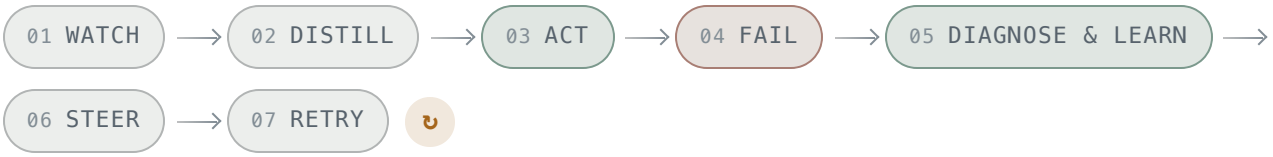
A skill that watches itself fail.

One YouTube tutorial, distilled into a checkpoint plan by an agent that only *watched*. Handed to a second agent to actually drive Audacity on a real noisy recording. It broke, four times, in three distinct ways — and each break became evidence, folded straight back into the plan that caused it.

Target file noisy_poem.wav Tutorial youtube.com/shorts/g3W3zy9ENCs

Model holo3-1-35b-a3b Runs so far 4



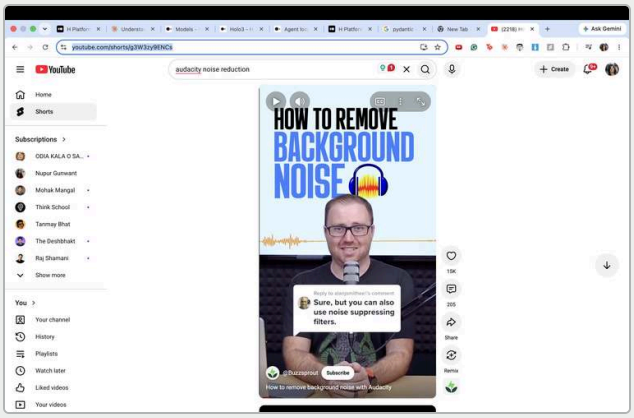


6 scripts built 4 live runs

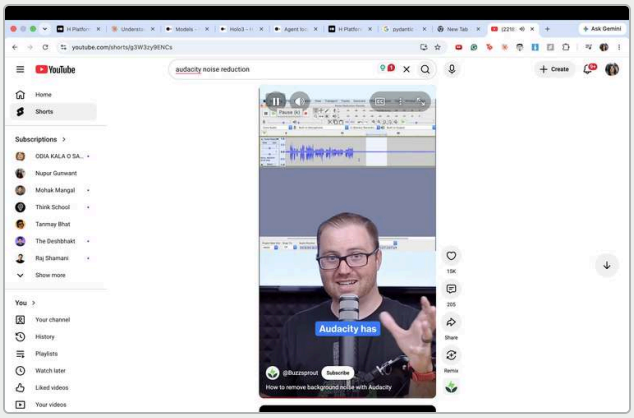
01 · watch Watch the tutorial, not the transcript

A video-watching agent opens the real tutorial in the user's own, already-signed-in Chrome — no scraped captions. It scrubs the seek bar, pausing every so often the way

a person previewing a tutorial would, reading the frame: what's clicked, what dialog is open, what changed.



What it opened — a 40-second Short on Audacity's Noise Reduction effect.



One of the frames it paused on — the demonstrator's own Audacity, mid-selection.

session **f1abe267…** sampled **~10** points across the timeline

distilled by **watching**, not transcript

02 · distill

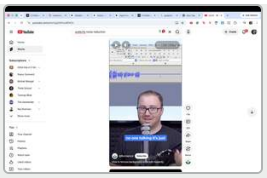
Turn viewing into a checkpoint, not a script

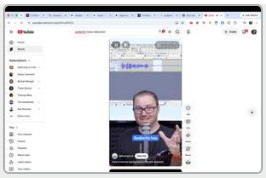
Each observed action becomes a milestone with three parts: an instruction (one way to get there), and a verify condition (the actual goal). The distinction matters later — the instruction can be wrong about the exact click; the verify condition can't.

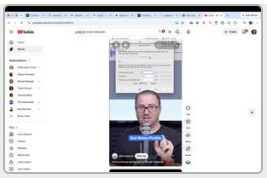

```
{
  "milestone": "Select noise profile section",
  "instruction": "Select a few seconds of audio that contains only background n
  "verify": "A portion of the waveform is highlighted/selected in the audio tra
  "irreversible": false,
  "screenshot": "obs_002.png"
}
```


plan_local.json · 5 milestones, one per demonstrated step


1 · select


2 · profile


3 · configure

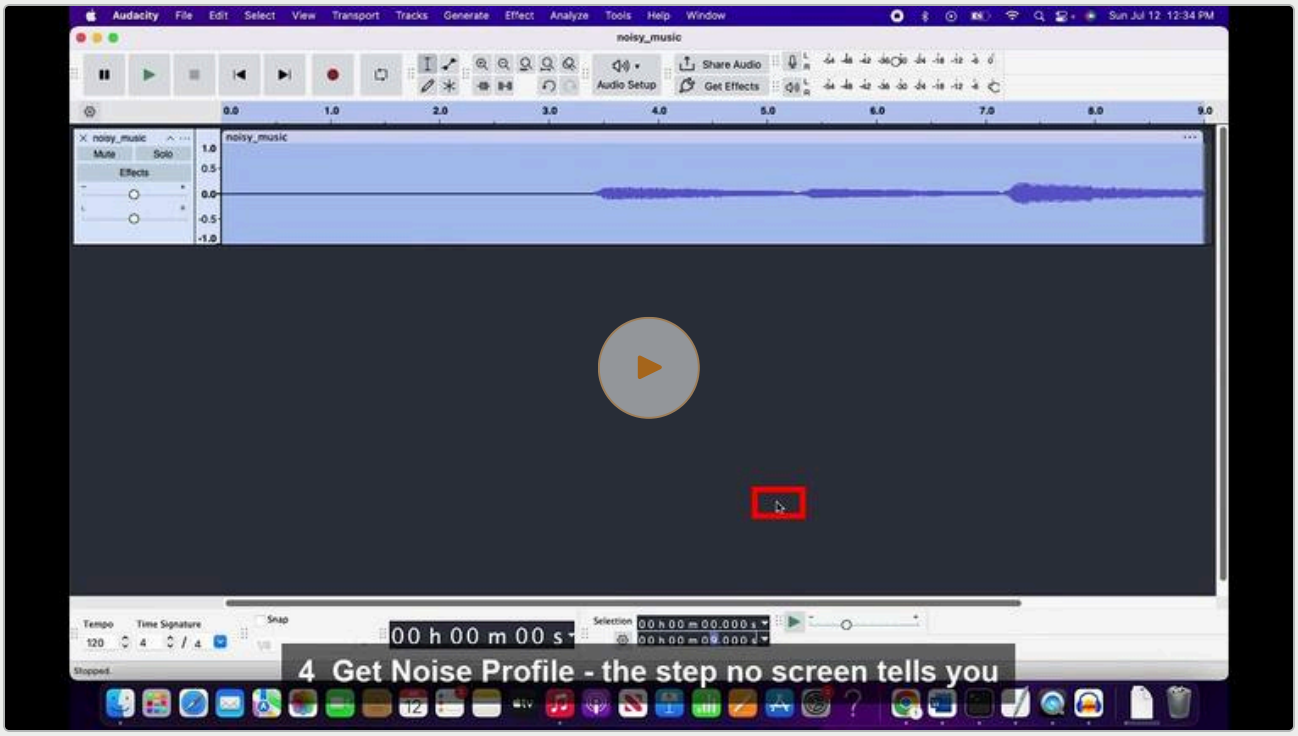

5 · apply

```
# youtube_to_local_skill.py — the watch → distill call
plan = watch_locally(video_url, goal)          # scrubs, screenshots, never rea
shots = save_screenshots(plan["session_id"], out_dir)
correlate(plan, shots)                          # attaches each step's real fra
```

03 · act

A second agent takes the wheel — for real

The plan, plus its real reference screenshots, gets handed to an independent desktop agent. It takes over the actual mouse, keyboard and screen on the user's own Mac, opens the real Audacity, and works the real noisy_poem.wav.



The real desktop, mid-run — watch it live. "Holo Agent Demo: Team ReeLearn" · 1:20 · opens on YouTube

Press play — the same technique, before and after, on a related run (a teammate's separate script, a different file, completed end to end):

noisy_music.wav — before

0:00 / 0:00

noisy_music_noise_removed.wav — after

0:00 / 0:00

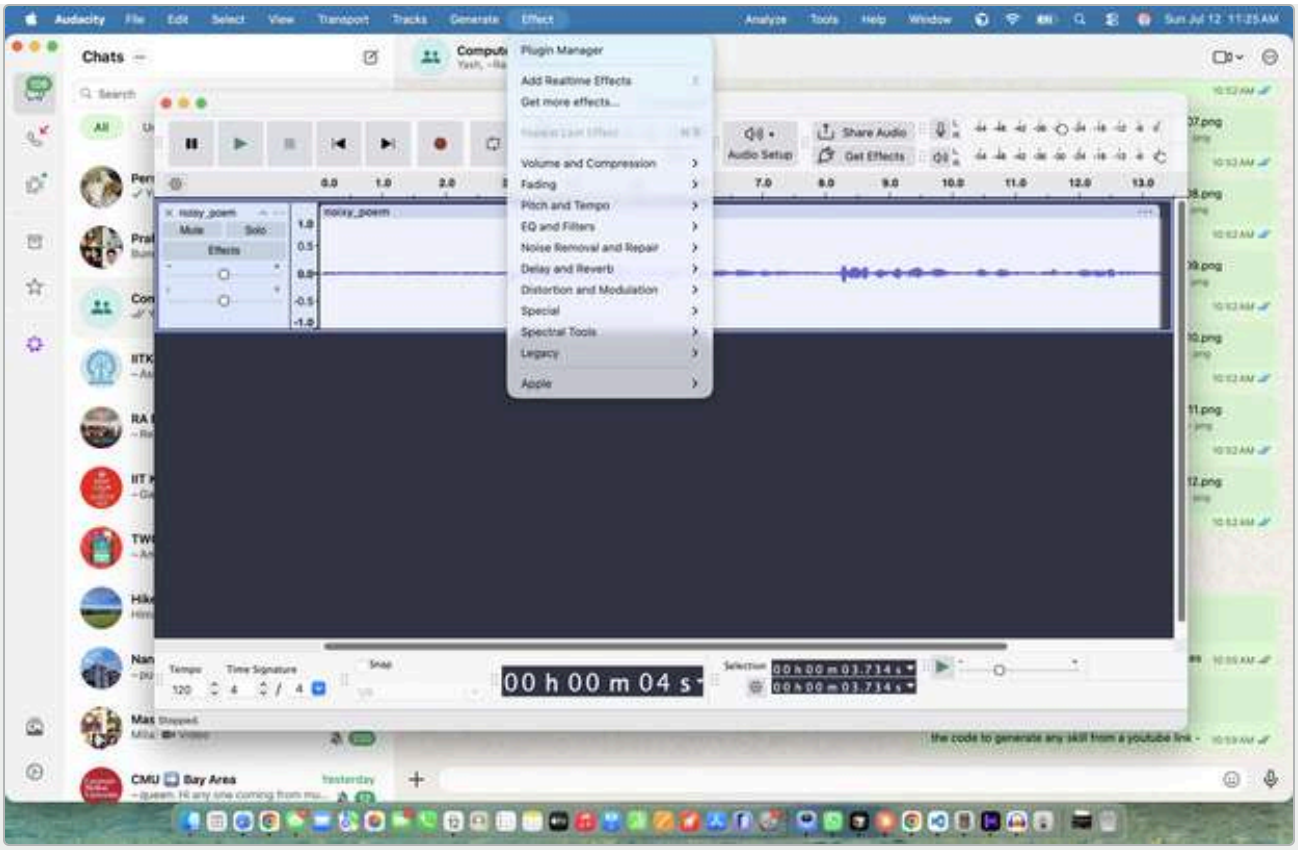
```
# fix_noisy_poem.py — non-destructive by construction
OUTPUT_WAV = "noisy_poem_cleaned.wav"  # never the original

message = UserMessageEvent(
    message=plan_to_task_text(plan),   # the checkpoint plan, in prose
    images=collect_reference_images(plan)  # its real screenshots, as pixels
)
```

04 · fail

It clicked "Effect." Its own screen answered.

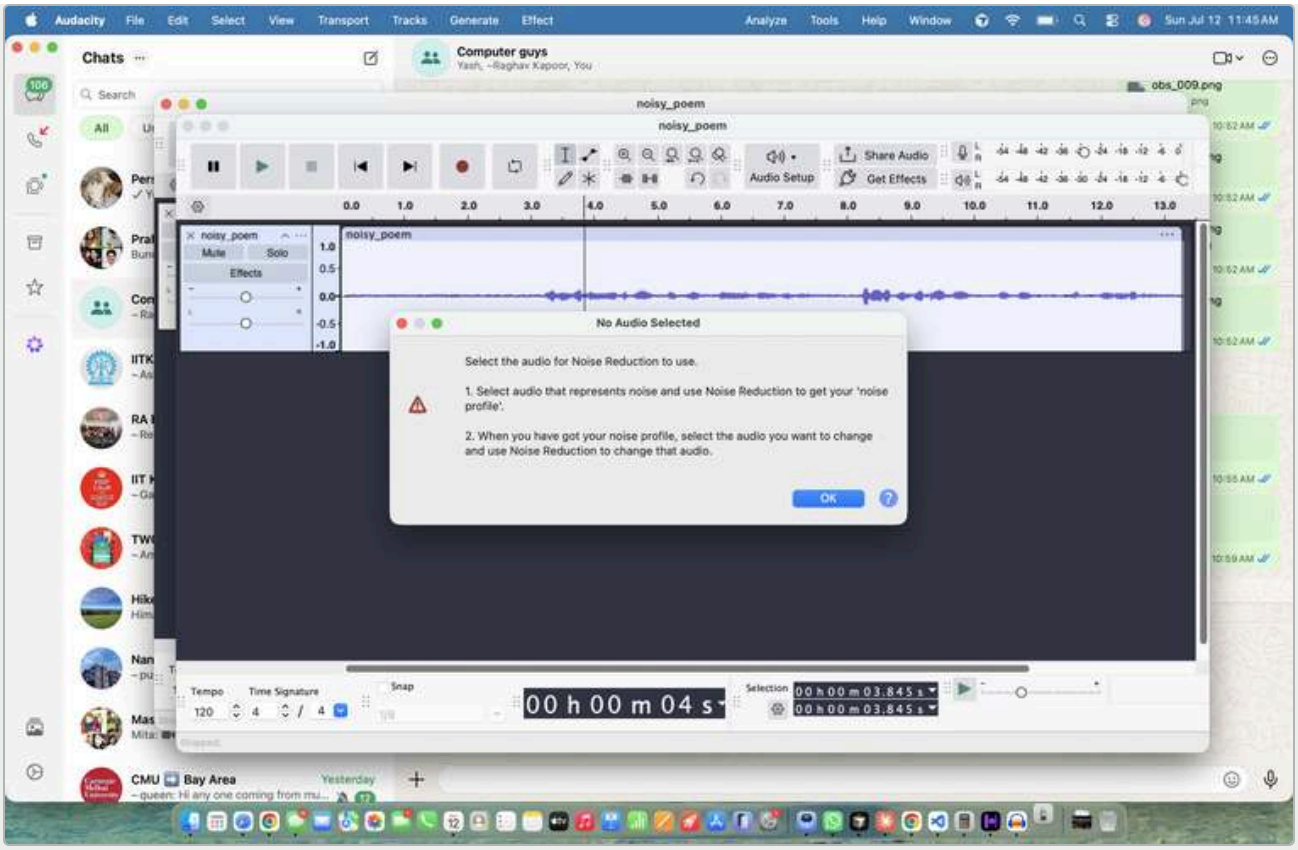
On macOS, the top menu bar always belongs to whichever app is frontmost — not necessarily the one you think you clicked into. Something on the same machine kept stealing focus mid-run.



`Expected — Effect menu open, Noise Removal and Repair right there. One click later, focus had silently moved to a different app entirely.`

"The click on 'Noise Removal and Repair' caused the screen to switch to another app instead."

A second, still-unsolved failure, found in three of four runs: the selection timer and the waveform both show a real selection — yet Noise Reduction refuses to see it.



`Selection timer reads 03.845s. Noise Reduction disagrees.` `still open`

"The dialog says 'No Audio Selected' but the selection timer clearly shows 03.845s."

`05 · diagnose & learn`

Ask the failed session what it saw — then rewrite the plan

The insight: the click didn't miss — it landed in a different app entirely. Audacity's menu bar only exists while Audacity is frontmost, and something else silently took focus first.

```
# the code that found it — diagnose_session.py
$ python diagnose_session.py ae26abda…

[04] 2026-07-12T18:25:45Z
  "the click caused the screen to switch
  to a different app instead of opening
  the submenu in Audacity."
```

Before → after the skill itself:

```
before:  steps 1–5   select → profile → configure → preview → apply

after:   + step 6  "Guard against losing focus to another app"
           verify: menu bar reads 'Audacity', not another app
           screenshot: ae26abda…/obs_004.png  ← the failure, as proof
```

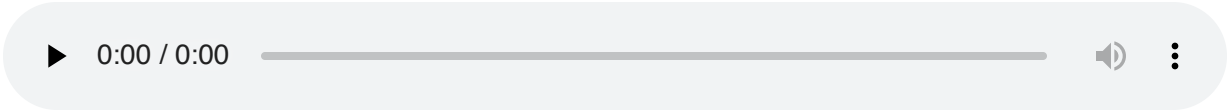
06 · steer Or just say "stop, look here" — live

Pause and resume are server-side calls, addressed by session ID alone — independent of whichever process is holding the desktop bridge open. Watching the real screen, a human can redirect a run mid-flight without waiting for it to time out first.

```python
# steer_session.py
client.sessions.pause_session(session_id)
client.sessions.send_session_messages(session_id,
    request=UserMessage(message="stop — second Audacity window behind this one"
client.sessions.resume_session(session_id)
```

This is the same mechanism as stage 05, just live instead of after the fact: a diagnosed failure and a human's steering message are both just evidence. Either one can be folded back into the plan. That's the actual gradient — not a sudden fix, a slope upward, one run at a time, whether the correction comes from the agent noticing its own screenshot or from a person watching over its shoulder.

the steering message, spoken — generated with Gradium TTS, same voice a narrated demo would use



07 · retry The loop bends back to "act"

Four live runs. Each one removed exactly one class of failure and exposed the next. Nothing here claims a finished success — the honest state of the loop, right now:

RUN	STEPS	DURATION	WHAT IT HIT	STATUS
ae26abda…	37	411.8s	Focus stolen by another app	found
2bd78317…	32	369.9s	Same, cmd-tab loop, ran out of budget	found
4c39d50c…	31	413.1s	Focus fixed — hit duplicate windows + phantom selection	focus solved
c9ef05b7…	37	413.9s	Windows fixed — phantom selection persists, 4th time	still open

Not every countermeasure survives contact: an anti-stuck-loop guard skill (`register_unstuck_skill.py`) was tried, then deliberately switched back off when it slowed runs down without preventing this specific failure — worth building, not worth keeping on by default. Next experiment queued instead: drive the selection through Audacity's own numeric Start/End time fields, a keyboard path, instead of a synthetic mouse drag on the waveform canvas.

Not a finished pipeline. A loop that gets a little more honest about itself with every pass.

H Company · holo3-1-35b-a3b · Audacity 3.7.8 · macOS desktop bridge